

ORBITAL

Autonomous Asteroid Mining Mission Operating System

Master Project Documentation Bundle

README.md

ORBITAL

Autonomous Asteroid Mining Mission Operating System

ORBITAL is a C++-based mission simulation platform for modeling autonomous robotic asteroid mining operations in deep space environments near Mars and the asteroid belt.

The platform combines:

- mission simulation
- robotics behavior systems
- orbital computation
- resource extraction modeling
- telemetry visualization
- autonomous navigation
- custom scenario scripting via `.rox` files

The project is designed as:

- a serious engineering portfolio project
 - a hackathon-ready technical demonstration
 - a scalable foundation for future aerospace simulation systems
-

Core Vision

Future autonomous mining fleets will require:

- predictive mission planning
- resilient autonomous behavior
- distributed rover coordination
- fuel optimization
- orbital timing computation

- mining efficiency analysis

ORBITAL acts as the software command layer for these systems.

Key Features

Phase 1 (Hackathon MVP)

Simulation Engine

- Mission runtime loop
- Asteroid environment model
- Rover movement
- Battery simulation
- Mining cycle simulation
- Resource collection
- Telemetry updates

.ROX Mission Files

Custom domain-specific scripting format.

Example:

```
MISSION TitanBelt

ASTEROID AX-44
POSITION 400 220
RESOURCE platinum
DENSITY 0.76

ROVER RX-7
POSITION 40 60
BATTERY 100
TASK mine
TARGET AX-44
```

GUI

- Mission map
- Rover tracking
- Telemetry panel
- Mining statistics
- Mission timeline

- Playback controls

Export System

- Save mission state
 - Export replay
 - Share `.rox` scenarios
-

Phase 2 (Future)

- Real orbital mechanics
 - AI optimization
 - Swarm rover coordination
 - Reinforcement learning
 - Real NASA data integration
 - Distributed simulation
 - GPU acceleration
-

Tech Stack

Backend

- C++20
- CMake
- EnTT ECS
- Eigen
- nlohmann/json
- fmt
- spdlog

Frontend

- Dear ImGui
- OpenGL
- GLFW

Tooling

- Git
- GitHub Projects
- CLion or VSCode
- RenderDoc
- Tracy Profiler

High-Level Architecture

```
[ .rox Mission Files ]  
    ↓  
[ Parser + Validator ]  
    ↓  
[ Simulation Engine ]  
    ↓  
[ ECS Runtime ]  
    ↓  
[ Physics + AI + Mining Systems ]  
    ↓  
[ Telemetry Pipeline ]  
    ↓  
[ GUI Renderer ]  
    ↓  
[ Export / Replay System ]
```

Folder Structure

```
orbital/  
├─ assets/  
├─ docs/  
├─ external/  
├─ shaders/  
├─ scenarios/  
├─ src/  
│   ├─ core/  
│   ├─ engine/  
│   ├─ ecs/  
│   ├─ gui/  
│   ├─ math/  
│   ├─ parser/  
│   ├─ physics/  
│   ├─ renderer/  
│   ├─ simulation/  
│   ├─ telemetry/  
│   └─ utils/  
└─ tests/
```

```
|— CMakeLists.txt
|— README.md
```

Build

```
mkdir build
cd build
cmake ..
cmake --build .
```

Hackathon Demo Goal

A live simulation where:

- multiple rovers navigate an asteroid field
- mining operations occur autonomously
- telemetry updates in real time
- resources are extracted and transported
- users can load/share `.rox` missions

The demo must visually communicate:

- systems engineering
 - autonomy
 - scalability
 - aerospace realism
-

PRD.md

Product Requirements Document

Product Name

ORBITAL

Problem Statement

Humanity will require autonomous off-world resource extraction systems to support:

- space manufacturing
- orbital infrastructure
- fuel production
- deep space exploration

Current educational and research tools are either:

- inaccessible
- highly specialized
- visually weak
- not robotics-oriented

ORBITAL solves this by providing a modular simulation platform for autonomous asteroid mining missions.

Product Goals

Primary Goals

- 1. Build a believable asteroid mining simulation platform**
 - 2. Demonstrate advanced systems engineering capability**
 - 3. Create reusable engine architecture**
 - 4. Teach advanced C++ through real implementation**
 - 5. Deliver a visually impressive hackathon demo**
-

User Personas

Robotics Engineer

Needs:

- mission simulation
- rover coordination
- telemetry visualization

Aerospace Student

Needs:

- orbital experimentation
- educational simulation
- system visualization

Hackathon Judge

Needs:

- visual clarity
 - technical depth
 - originality
-

Functional Requirements

Mission Loader

- Load `.rox` files
- Validate syntax
- Create runtime entities

Simulation Engine

- Fixed timestep loop
- Entity updates
- Event handling
- Resource tracking

Rover System

- Navigation
- Mining
- Battery drain
- Failure states

Asteroid System

- Resource density
- Structural depletion
- Terrain zones

Telemetry

- Rover stats
- Mining rate
- Fuel usage
- Signal strength

GUI

- Mission control interface
- Interactive map
- Playback controls
- Statistics panels

Export

- Save mission
 - Export logs
 - Replay simulation
-

Non-Functional Requirements

Performance

- 60 FPS minimum
- Support 100+ entities

Reliability

- Parser error recovery
- Stable runtime

Extensibility

- Modular systems
- Plugin-capable architecture

Maintainability

- Clear subsystem separation
 - Documentation-first approach
-

Success Metrics

Hackathon Success

- Simulation runs without crashes
- Judges understand project quickly
- Strong visual presentation

Technical Success

- ECS architecture complete
- Parser operational
- Replay system functional

Educational Success

- Learn advanced modern C++
 - Understand engine architecture
 - Understand simulation design
-

TRD.md

Technical Requirements Document

Core Technical Architecture

Engine Paradigm

Entity Component System (ECS)

Reason:

- scalable
 - cache-friendly
 - modular
 - standard in simulation/game engines
-

Runtime Loop

```
while(running)
{
    processInput();
    fixedUpdate();
    render();
}
```

Major Systems

1. Parser System

Responsibilities:

- tokenize `.rox`
- syntax validation
- AST generation
- runtime configuration

Components

- Lexer
- Parser
- Validator
- Serializer

2. ECS Runtime

Entities

- rovers
- asteroids
- stations
- drones

Components

- Transform
- Battery
- Velocity
- MiningState

- Telemetry
- Navigation

Systems

- MovementSystem
 - MiningSystem
 - PowerSystem
 - AIBehaviorSystem
 - TelemetrySystem
-

3. Simulation Layer

Responsibilities:

- asteroid depletion
 - mining output
 - mission state
 - collision zones
 - communication ranges
-

4. Renderer

MVP Scope

2D top-down visualization.

Reason: 3D dramatically increases complexity.

Future: Upgrade to orbital 3D rendering.

5. Telemetry Pipeline

Responsibilities:

- live statistics
 - event history
 - mission logs
 - replay snapshots
-

Data Flow

```
.rox File
↓
Lexer
↓
Parser
↓
AST
↓
Mission Builder
↓
ECS Runtime
↓
Simulation Systems
↓
Telemetry
↓
GUI Rendering
```

Performance Targets

Metric	Target
FPS	60
Startup Time	< 3 sec
Mission Load	< 1 sec
Entity Count	100+

C++ Concepts To Learn

Core

- RAII
- move semantics
- smart pointers
- templates
- constexpr

- lambdas

Advanced

- multithreading
 - allocators
 - ECS memory layouts
 - event buses
 - custom serialization
 - profiling
-

Engineering Risks

Risk 1: Scope Explosion

Mitigation:

- stay 2D
- no networking
- no multiplayer

Risk 2: Graphics Complexity

Mitigation:

- use Dear ImGui
- minimal rendering pipeline

Risk 3: Parser Instability

Mitigation:

- strict grammar
 - incremental parser testing
-

APPFLOW.md

Application Flow

Startup Flow

```
Launch Application
  ↓
Initialize Renderer
  ↓
Initialize ECS
  ↓
Load Assets
  ↓
Display Main Dashboard
```

Mission Flow

```
User Opens .rox File
  ↓
Parser Validates Mission
  ↓
Mission Builder Creates Entities
  ↓
Simulation Starts
  ↓
Telemetry Updates
  ↓
User Controls Playback
  ↓
Mission Ends
  ↓
Export Results
```

Rover Operational Flow

```
Receive Task
  ↓
Navigate To Target
  ↓
Scan Resource Zone
  ↓
```

```
Begin Mining
  ↓
Consume Power
  ↓
Store Resources
  ↓
Transmit Telemetry
  ↓
Return To Base
```

Error Handling Flow

```
Parser Error
  ↓
Highlight Faulty Line
  ↓
Show Human-Readable Error
  ↓
Prevent Simulation Start
```

Replay Flow

```
Simulation Snapshot
  ↓
Store Frame Data
  ↓
Serialize Replay
  ↓
Replay Engine
  ↓
Playback Visualization
```

UI_UX_DESIGN_BRIEF.md

UI/UX Design Brief

Design Philosophy

The interface should feel like:

- mission control software
- aerospace telemetry systems
- industrial operations dashboards

Avoid:

- cartoon visuals
- excessive gradients
- gaming aesthetics

Target: Professional engineering software.

Visual Style

Colors

Primary

- black
- graphite
- deep blue

Accent

- cyan
- orange
- white telemetry indicators

Warning

- amber
 - red
-

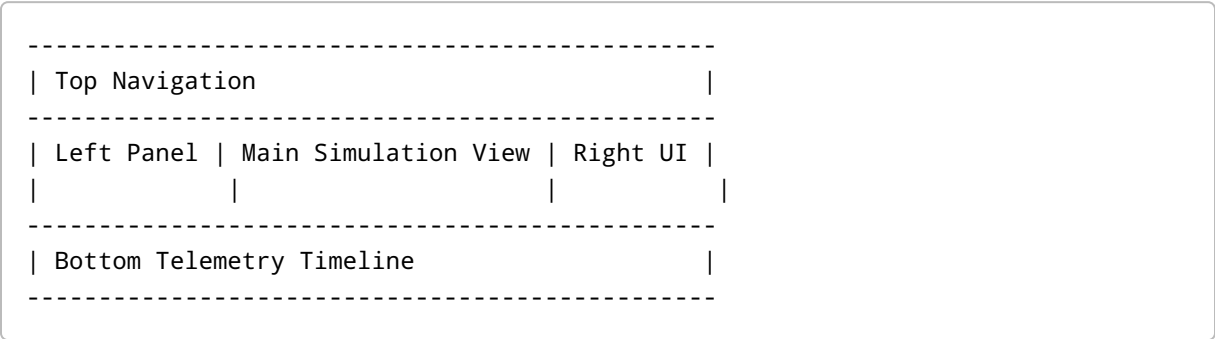
Typography

Use:

- JetBrains Mono
- Inter

Reason: Technical readability.

Layout Structure



Core Screens

1. Dashboard

Displays:

- mission overview
- active rover count
- alerts
- simulation controls

2. Mission Viewer

Displays:

- asteroid map
- rover paths
- mining zones
- signal ranges

3. Telemetry Panel

Displays:

- battery

- velocity
 - cargo
 - resource yield
-

4. Mission Timeline

Displays:

- events
 - alerts
 - mining milestones
-

UX Rules

Rule 1

Every major event must be visible.

Rule 2

User should never lose mission context.

Rule 3

Controls must remain minimal.

Rule 4

The simulation itself is the hero.

Animation Guidelines

Minimal animations only:

- rover movement
- telemetry pulse
- signal transmission

Avoid excessive UI motion.

BACKENDSCHEMA.md

Backend Schema

Core Runtime Structures

Rover Entity

```
struct Rover
{
    EntityId id;
    std::string name;
    Vector2 position;
    float battery;
    float cargo;
    TaskType task;
};
```

Asteroid Entity

```
struct Asteroid
{
    EntityId id;
    Vector2 position;
    ResourceType resource;
    float density;
    float remainingMass;
};
```

Mission State

```
struct MissionState
{
    float elapsedTime;
    int activeRovers;
```

```
    float totalYield;  
};
```

ECS Components

TransformComponent

```
struct TransformComponent  
{  
    Vector2 position;  
    float rotation;  
};
```

BatteryComponent

```
struct BatteryComponent  
{  
    float charge;  
    float drainRate;  
};
```

MiningComponent

```
struct MiningComponent  
{  
    bool active;  
    float extractionRate;  
};
```

Event System

Events

```
RoverBatteryLow  
MiningStarted  
MiningCompleted  
CollisionDetected  
MissionCompleted
```

.ROX Grammar

Example

```
MISSION MarsBelt  
  
ROVER RX-1  
POSITION 40 50  
BATTERY 100  
TASK MINE  
TARGET AX-4  
  
ASTEROID AX-4  
POSITION 500 300  
RESOURCE IRON  
DENSITY 0.83
```

Serialization Plan

Save Formats

Human-readable

`.rox`

Runtime Replay

`.orb` (binary replay snapshots)

Replay Snapshot Schema

```
struct ReplayFrame
{
    float timestamp;
    std::vector<EntityState> entities;
};
```

IMPLEMENTATION_PLAN.md

Implementation Plan

Total Budget

Approximately 300 hours.

PHASE 1 — Foundation

Duration: 40 Hours

Goals

- project setup
- rendering window
- CMake
- ECS initialization
- logging
- folder structure

Deliverables

- OpenGL window
 - Dear ImGui running
 - EnTT integrated
 - GitHub repository initialized
-

PHASE 2 — .ROX Parser

Duration: 45 Hours

Goals

- lexer
- parser
- validation system
- mission loading

Deliverables

- `.rox` syntax operational
 - parser error messages
 - AST generation
-

PHASE 3 — ECS Simulation Core

Duration: 60 Hours

Goals

- rover systems
- asteroid systems
- battery systems
- mining simulation

Deliverables

- moving rovers
 - active mining
 - telemetry updates
-

PHASE 4 — GUI + Visualization

Duration: 55 Hours

Goals

- mission map
- telemetry panels

- mission controls
- statistics

Deliverables

- fully interactive GUI
 - live simulation rendering
-

PHASE 5 — Replay + Export

Duration: 35 Hours

Goals

- replay snapshots
- export system
- save/load missions

Deliverables

- replay viewer
 - exportable files
-

PHASE 6 — Polish + Demo Stability

Duration: 40 Hours

Goals

- optimization
- bug fixing
- visual polish
- stress testing

Deliverables

- stable demo build
 - presentation-ready system
-

PHASE 7 — Hackathon Presentation Assets

Duration: 25 Hours

Goals

- cinematic demo scenario
- README cleanup
- architecture diagrams
- slides

Deliverables

- final pitch assets
 - demo recording
 - screenshots
-

Weekly Execution Strategy

Week 1

Engine foundation.

Week 2

Parser + ECS runtime.

Week 3

Simulation systems.

Week 4

GUI systems.

Week 5

Replay/export.

Week 6

Polish and presentation.

Critical Scope Rules

DO NOT ADD

- multiplayer
- networking
- cloud systems
- full orbital mechanics
- procedural planets
- machine learning
- 3D rendering

These destroy timeline viability.

MVP Definition

The MVP is complete when:

- `.rox` missions load
- rovers move
- mining occurs
- telemetry updates
- GUI visualizes operations
- replay/export works

Everything else is secondary.

Hackathon Demo Script

Demo Sequence

1

Open ORBITAL dashboard.

2

Load asteroid mission.

3

Simulation starts.

4

Rovers navigate autonomously.

5

Mining operations begin.

6

Battery drains and telemetry updates.

7

Resource extraction statistics increase.

8

Replay mission.

9

Export `.rox` mission.

ORBITAL-LITE EDITION

Lightweight Execution Variant

ORBITAL-LITE is the optimized implementation target for mid-range laptops and fast hackathon execution.

ORBITAL-LITE Constraints

Target Hardware

- 8 GB RAM
- Integrated GPU
- Dual-core or low-power CPUs

Reduced Scope

Included

- 2D simulation
- ECS runtime
- `.rox` mission parser
- rover navigation
- mining systems
- telemetry dashboard
- replay system
- export/import

Excluded

- 3D rendering
- realistic orbital mechanics
- networking
- AI model inference
- multiplayer
- cloud systems
- GPU compute

ORBITAL-LITE Technology Stack

Core

- C++20
 - CMake
 - Ninja
 - OpenGL
 - GLFW
 - Dear ImGui
 - EnTT ECS
 - Eigen
 - spdlog
 - fmt
-

Recommended Development Environment

IDE

CLion

Build System

Ninja

Performance Tools

- ccache
- Tracy Profiler

Recommended Arch Linux Setup

```
sudo pacman -S cmake ninja gcc gdb git ccache  
sudo pacman -S glfw-x11 mesa mesa-utils
```

CLion Build Configuration

Toolchain

Use:

- GCC
- Ninja
- Debug build during development

Enable ccache

Inside CLion:

```
Settings → Build, Execution, Deployment → CMake
```

Add:

```
-DCMAKE_CXX_COMPILER_LAUNCHER=ccache
```

WakaTime Integration

Install WakaTime CLI

```
yay -S wakatime
```

OR:

```
pip install wakatime
```

Install CLion Plugin

Inside CLion:

```
Settings → Plugins → Marketplace
```

Search:

```
WakaTime
```

Install and restart CLion.

Authenticate

After restart:

```
Tools → WakaTime → Login
```

Paste API key from:

<https://wakatime.com/settings/api-key>

Recommended ORBITAL-LITE Folder Structure

```
orbital-lite/  
├── assets/  
├── scenarios/  
├── shaders/  
├── docs/  
├── src/  
│   ├── core/  
│   ├── ecs/  
│   ├── gui/  
│   ├── parser/  
│   ├── renderer/  
│   ├── simulation/  
│   ├── telemetry/  
│   └── utils/  
├── tests/  
├── CMakeLists.txt  
└── README.md
```

ORBITAL-LITE MVP Goal

The MVP is successful when:

- `.rox` files load successfully
- rovers move autonomously
- asteroids can be mined
- telemetry updates live
- replay system functions
- GUI remains stable at 60 FPS

Suggested Development Order

Step 1

Window + Dear ImGui.

Step 2

Basic renderer.

Step 3

ECS runtime.

Step 4

`.rox` parser.

Step 5

Simulation systems.

Step 6

Telemetry panels.

Step 7

Replay/export.

Step 8

Polish and demo assets.

Critical Engineering Principle

Build:

- small
- modular
- stable

- extensible

Do not build:

- massive
 - cinematic
 - overengineered
 - unstable
-

Final Engineering Principle

Do not build:

- a toy
- a game
- a UI demo

Build:

- a simulation platform
- an extensible engine
- a believable mission operating system

That distinction changes every architectural decision.